

Match Statement

Cos'è il match Statement in Python

Il **match statement** è una funzionalità introdotta in Python 3.10 che permette di eseguire operazioni basate sulla corrispondenza di pattern (pattern matching). È particolarmente utile quando si devono gestire diversi casi in modo chiaro e leggibile, evitando lunghe catene di `if`, `elif`, `else`.

Il match statement funziona confrontando il valore di una variabile con una serie di pattern (casi).

Se un caso corrisponde, viene eseguito il blocco di codice associato. Se nessun caso corrisponde, viene eseguito il caso di default (`case _`).

In altre parole Il match statement è ideale quando devi gestire molti casi distinti basati sul valore di una variabile.

Supponiamo di stare a una festa di simulare il gioco di Mario Kart, dobbiamo fare 3 giri di una pista e finito il terzo giro ogni giocatore finisce il gioco e viene visualizzata la classifica.

Vogliamo di raccogliere le posizioni finali: se l'utente inserisce la posizione uno il programma la ritorna in posizione cardinale (`1st`) e così via.

Considera l'utente inserisca i numeri 1,2,3 e ci deve stampare i numeri cardinali `1st` , `2nd` , `3rd` , dal 4 in poi si usa il `th`



Python

```
1  n = int(input("Inserire un numero:"))
2
3
4
5  if n == 1:
6
7      print(f"{n}st!")
8
9  elif n == 2:
10
11      print(f"{n}nd!")
12
13  elif n == 3:
14
15      print(f"{n}rd!")
16
```

```

17 elif n == 4:
18
19     print(f"{n}th!")
20
21 else :
22
23     print(f"{n}th!")

```

In questo esempio sono stati valutati tutti i valori possibili di `n` e per ogni valore veniva stampato il messaggio con la nomenclatura cardinale. Questo ovviamente diventa complicato quando scriviamo righe e righe di codice, in questo caso viene in aiuto il match statement:

è un metodo che **migliora la leggibilità del codice e la comprensione e fa parte della buona pratica di programmazione**.

Funzionamento e sintassi del Match Statement

Si parte da una variabile con un determinato valore, e si prende questa variabile si fanno vari casi e per ogni caso deve eseguire una determinata istruzione.

Se nessun case è vero allora si passa al case di default (`case _`).

In parole povere la sintassi del match case è:



Python

```

1 match variabile:
2     case pattern1:
3         # Blocco di codice da eseguire se variabile corrisponde a
        pattern1
4     case pattern2:
5         # Blocco di codice da eseguire se variabile corrisponde a
        pattern2
6     case pattern3 (opzionale(if condizione)):
7         # Blocco di codice da eseguire se variabile corrisponde a
        pattern3 E la condizione è vera
8     case _:
9         # Blocco di codice da eseguire se nessun caso corrisponde
        (caso di default)

```

Componenti della sintassi

1. `match variabile:`

- **Inizia il match statement specificando la variabile che vuoi confrontare con i vari pattern** (ad esempio: nell'[esercizio](#) sopra la variabile da confrontare è `n`, di conseguenza scriverò `match n`, dove `n` è la variabile da confrontare).

1. `case pattern:` :

- **Definisce un caso specifico.**

Se il valore della variabile corrisponde al pattern, viene eseguito il blocco di codice associato (Ad esempio: nell'[esercizio](#) sopra al posto di `if n==1:` scriverò `case 1:` perché corrisponde al caso in cui la variabile è uguale a `1`).

1. [if condizione](#):

- si può usare i conditionals statements con i match case, questo aspetto verrà approfondito più avanti.

1. `case _:` :

- **Rappresenta il caso di default, che viene eseguito se nessun altro caso corrisponde.**

In parole povere `case _:` gestisce tutti i casi non coperti dai pattern precedenti.

Quindi [l'esercizio](#) può essere riscritto così:



Python

```
1  match n:
2      n = 1
3      case 1:
4          print(f"{n}st!")
5      n = 2:
6      case 2:
7          print(f"{n}nd!")
8      n = 3:
9      case 3:
10         print (print(f"{n}rd!"))
11     n = 4:
12     case 4:
13         print(f"{n}th!")
14     #default case
```

```

15         case _:
16             print(f"{n}th!")
17

```

Come si può vedere da questo esempio il match abbiamo dichiarato il `match n` all'inizio del blocco di codice del match statement.

In seguito abbiamo definito i `case pattern` in base al valore della variabile che vogliamo confrontare.

Per ultimo abbiamo impostato il caso di default `case _:` : se tutti gli altri pattern non sono veri allora il flusso analizza il caso di default ed esegue l'azione che corrisponde a quel caso.

Il match statement può essere usato in qualsiasi contesto (match annidati, `if`, `elif`, `else`, `while` e `for loop`).

Match statements e OR(|)

Si può usare l'operatore `|` (OR) per combinare più pattern in un singolo caso. Questo è utile quando vogliamo gestire più valori nello stesso modo.

Gestione di codici di stato HTTP

Supponiamo di avere uno stato HTTP da gestire.

Posso confrontare più valori associati a quello stato in 2 modi:

1. Primo Modo:



Python

```

1  #http status (if/elif/else version)
2  http_status = 200
3  if http_status ==200 or http_status ==201:
4      print("success")
5  elif http_status==400:
6      print("Bad Request")
7  elif http_status==500 OR http_status==501
8      print("Server Error")
9  case _:
10     print("Unknown")

```

1. Secondo modo (Match statement):



Python

```
1 http_status = 200
2 match http_status:
3     case 200|201:
4         print ("Success")
5     case 400:
6         print("Bad Request")
7     case 404:
8         print("Not Found")
9     case 500|501:
10        print("Server Error")
11    case _:
12        print ("Unknown")
```

✎ Da notare come in questo caso l'OR è tradotto con il simbolo `|`, questo perché nel match statement se scrivo `or` Python mi dà errore.

Inoltre l'uso di `|` rende il codice più compatto e leggibile rispetto a una catena di `if-elif`.

Il match statement e le condizioni

Puoi aggiungere condizioni specifiche a un caso usando `if`.

Questo ti permette di gestire casi più complessi.

Classificazione di numeri in base a proprietà



Python

```

1  #insert a number
2  n = int(input("Insert a number"))
3  match n:
4      case n if n>0 and n%2==0: #in questo caso l'if diventa sia: "se
       il caso n sia maggiore di zero e pari"
5          print(f"{n} is positive and even")
6      case n if n>0 and n%2!=0:
7          print(f"{n} is postive and odd")
8      case n if n<0 and n%2==0:
9          print(f"{n} is negative and even")
10     case n if n<0 and n%2!=0:
11         print(f"{n} is negative and odd")
12     case _:
13         print("The inserted number is 0")

```

Analizzando questo esempio possiamo vedere come l'utilizzo di `if` sia utile. In particolare:

- `case n if condizione` :
Puoi usare `if` per aggiungere condizioni specifiche a un caso.
- **Flessibilità:**
Questo approccio è molto flessibile e ti permette di gestire casi complessi senza dover scrivere codice ridondante.
Il `match` si usa solo quando la variabile assume diverse valori e seguendo questi valori devo far eseguire determinate azioni.

Visto così in realtà il `match` statement sembra essere un'alternativa più elegante e leggibile ai tradizionali **conditional statements** in Python, tuttavia sono due modi di procedere diversi:

1. Match Statements:

è particolarmente utile in situazioni specifiche dove il **pattern matching** (corrispondenza di pattern) semplifica notevolmente il codice e lo rende più intuitivo.

Il `match` statement è ideale quando devi gestire molti casi distinti basati sul valore di una variabile

2. Conditional Statements:

Utile quando le condizioni sono complesse e non si basano su pattern specifici (es: confronti tra più variabili, operazioni logiche intricate), i `conditional statements` tradizionali sono più adatti.

≡ Esempio Pratico



Python

```
1 x = 10
2 y = 20
3
4 if x > y:
5     print("x è maggiore di y")
6 elif x < y:
7     print("x è minore di y")
8 else:
9     print("x è uguale a y")
```

≡ In Conclusione

Usa il **match statement** quando:

- Devi gestire molti casi basati su valori specifici.
- Lavori con strutture dati complesse (liste, dizionari, tuple).
- Vuoi estrarre valori da strutture dati.
- Vuoi migliorare la leggibilità del codice.

Usa i **conditional statements tradizionali** quando:

- Le condizioni sono complesse e non basate su pattern.
- Devi supportare versioni di Python precedenti alla 3.10

Match Statement e variabili

Mettiamo sì abbiano 2 variabili:



Python

```
1 g = "f"
2 age = 5
3 match (g, age):
4     case ("f", 5):
5         print("Piccola!")
6     case ("m", 5):
```

```

7         print("Piccolo!")
8     case (f", 10):
9         print("Grande")
10    case ("m", 10):
11        print("Gigante")
12    case _:
13        print("Kinder Sorpresa!")

```

Quindi riprendendo dalla [definizione di match statements](#), in questo esempio una volta dichiarati le variabili `g` e `age` con i rispettivi valori, queste variabili vengono racchiuse in una tupla `g, age` e ogni `case` rappresenta un pattern specifico per la tupla.

Quindi ogni `case` definisce un pattern che la tupla `( g, age )` deve soddisfare:

1. **Primo** `case ( case (f, 5) )`:

- Questo caso corrisponde se `g == "f"` e `age == 5`.
- Se la tupla `(g, age)` è `("f", 5)`, viene eseguito il blocco di codice associato: `print("Piccola!")`.

2. **Secondo** `case ( case ("m", 5) )`:

- Questo caso corrisponde se `g == "m"` e `age == 5`.
- Se la tupla `(g, age)` è `("m", 5)`, viene eseguito il blocco di codice associato: `print("Piccolo!")`.

3. Terzo `case ( case ("f", 10) )`:

- Questo caso corrisponde se `g == "f"` e `age == 10`.
- Se la tupla `(g, age)` è `("f", 10)`, viene eseguito il blocco di codice associato: `print("Grande!")`.

4. **Quarto** `case ( case("m", 10) )`:

- Questo caso corrisponde se `g == "m"` e `age == 10`.
- Se la tupla `(g, age)` è `("m", 10)`, viene eseguito il blocco di codice associato: `print("Gigante!")`.

5. **Caso di default** `( case _ )`:

- Il caso di default `( case _ : )` viene eseguito se **nessuno dei casi precedenti** corrisponde.

- Rappresenta tutti i casi non coperti dai pattern specifici.
- Nel tuo esempio, se la tupla (g, age) non corrisponde a nessuno dei casi precedenti, viene eseguito il blocco di codice associato: `print("Kinder Sorpresa!")`.

Match Statement e liste

Puoi usare il match statement per confrontare liste e gestire casi specifici in base al loro contenuto o lunghezza fissa.

Gestione di ingredienti



Python

```
1 ingredienti = ["Pomodoro", "mozzarella", "basilico"]
2 match ingredients:
3     case ["pomodoro", "mozzarella", "basilico"]:
4         print("Puoi fare una Caprese!")
5     case ["pomodoro", "mozzarella", * _]:
6         print("Puoi fare la Pasta al pomodoro")
7     case ["Pane", "Prosciutto", "Formaggio"]:
8         print("Puoi fare un Panino!")
9     case _:
10        print("Non saprei cosa consigliare...sperimenta!")
```

Analizziamo questo esempio:

- `* _` :
Rappresenta "tutti gli elementi rimanenti".
Ad esempio, `case ["Pomodoro", "Mozzarella", * _]` corrisponde a una lista che inizia con "Pomodoro" e "Mozzarella", seguita da qualsiasi altro elemento. Quindi se ho una lista che inizia con gli elementi "Pomodoro" e "Mozzarella", mi riporterà subito al caso due.

Match statement e dizionari

Puoi usare il match statement per confrontare dizionari e gestire casi specifici in base alle loro chiavi e valori.

Gestione di utenti



Python

```

1  user = {"nome": "Luca", "ruolo": "admin"}
2  match user:
3  case ("nome": name, "ruolo": "admin"): #la variabile name serve
    per definire il fatto che voglio concentrarmi sul valore della
    chiave ruolo perchè non voglio porre l'attenzione sulla chiave
    nome.
4      print(f"Benvenuto amministratore {name}")
5  case ("nome": name, "ruolo": "utente")
6      print(f"Benvenuto utente {name}")
7  case _:
8      print ("Ruolo non riconosciuto")

```

Questo mi è utile quando voglio estrarre i valori da un dizionario e usarli nel blocco di codice associato al caso.

Ad esempio Ad esempio, `{"nome": name}` estrae il valore associato alla chiave "nome" e lo assegna alla variabile `name`.

⚡ **Da non confondere con l'altro metodo per estrarre i valori dei dizionari visto fino ad adesso**



Python

```

1  user = {"nome": "Luca", "ruolo": "admin"}
2  match user:
3      case ("nome": name["Luca"], "ruolo": "admin"):

```

In questo caso mi ritornerà errore perché, in Python, con il match case non si può fare

✓ **Success**



Python

```

1
2      print("Admin")
3  case ("nome": name, "ruolo": "admin"):
4      print("admin")

```

Match statement e tuple

Puoi usare il match statement per confrontare tuple e gestire casi specifici in base ai loro elementi.

Gestione di punti in un piano cartesiano



Python

```
1 point = (3,5)
2 match point:
3     case (0,0):
4         print("Origine")
5     case(x, 0):
6         print(f"Punto sull'asse x: ((x), 0)")
7     case (0, y):
8         print(f"Punto sull'asse y: (0,(y))")
9     case _:
10        print(f"Punto generico: (point)")
```

Analizziamo questo esempio

- **Estrazione di elementi:** Puoi estrarre elementi da una tupla e usarli nel blocco di codice associato al caso. Ad esempio, `case (x, 0)` estrae il primo elemento e lo assegna alla variabile `x`.
- **Pattern matching su tuple:** Puoi confrontare tuple di lunghezza fissa o variabile.